VICTR: Value-Informed Context Retrieval for Vision-Language-Action Models

Anonymous Authors

***Abstract*—Vision-language-action (VLA) models are increasingly capable generalists, but adapting a pretrained VLA to a new embodiment, environment, or task still typically requires collecting many demonstrations, mixing them with the original training data, and fine-tuning the base policy – an expensive loop that must be repeated for every new domain. In-context learning (ICL) offers an alternative – where a model's behavior is adapted at inference time by conditioning on examples supplied as part of its input, rather than by updating its weights. Prior work has shown that VLAs can be post-trained to use a handful of demonstrations placed directly in their context window (i.e. prepended as extra input tokens the policy attends to when predicting its next action, with no change to its weights), avoiding further gradient updates altogether. A central open question for in-context VLAs is what to retrieve into that context – i.e., which stored demonstration segment to select and place in the window for a given query – and how: existing methods rely primarily on visual similarity between the current observation and candidate demonstration frames. We argue that task-relative *progress* is an equally important and underexplored retrieval signal, and that value function estimators (VFEs) – widely used to provide reward signals for VLA reinforcement learning – can themselves be made in-context learners. We present three contributions: (1) the first In-Context Value Function Estimator (IC-VFE), which conditions on a full-length demonstration to produce calibrated progress estimates on in- and out-of-domain tasks without task-specific fine-tuning; (2) the use of estimated state value, fused with visual similarity via a learned scalar combination, as a retrieval signal for in-context VLA policies, extending prior work that relies on visual similarity alone; and (3) VICTR, a policy that composes both signals at inference time, together with a rigorous real-world evaluation across 44 tasks on a bimanual manipulator, complemented by a robustness ablation on the LIBERO-100 benchmark, ablating each design element against RICL, ICRT, and ICLR baselines.**

***Index Terms*—Vision-Language-Action (VLA) Models, In-Context Learning (ICL), Retrieval-Augmented Generation (RAG), Value Function Estimation, Reinforcement Learning, Bimanual Manipulation**

I. Introduction

a) Scalable deployment challenges for VLAs.: Vision-language-action models are one of the most promising current approaches to tackling generalist manipulation problems in the real world. Much of their success has come from exposing them to increasingly diverse datasets [1], [2], and reinforcement learning has recently been shown to boost their performance on specific tasks beyond the ceiling of imitation learning alone [2]. Yet no matter how large a pretraining dataset grows, there will always be something out of domain: a new embodiment, a new and complex environment, or a particular short- or long-horizon task the policy has never seen. Recent distributed real-world evaluations bear this out at scale: even strong generalist policies show a marked drop-off on long-horizon and out-of-distribution tasks once evaluated outside a single lab's task suite [3], underscoring the need for adaptation mechanisms that do not require a full retraining cycle per new domain. The standard remedy – collect a large set of demonstrations in the new domain, mix them with the original data to avoid catastrophic forgetting, and fine-tune – is slow and must be repeated for every new domain. Imitation learning alone is also typically insufficient to reach the throughput required of a robust VLA on a specific task, motivating on-policy practice with occasional human intervention [2]. The central difficulty is the end-to-end cost of adapting a VLA to a new domain: collect many demonstrations, retrain the base policy, collect many rollouts, and fine-tune again. This work targets a much shorter loop: collecting few demonstrations ($\leq$10, rather than hundreds), performing real rollouts in which those demonstrations are only ever supplied to a fixed-parameter policy as context – never used to update its weights – and requiring at most a single fine-tuning step to reach comparable results. We propose a bi-level retrieval policy with an in-context value function estimator to enable this loop.

b) In-context learning in VLAs.: In-context learning – supplying extra information in a transformer's context window to boost performance without updating its parameters – has already seen substantial success in the VLM world, where a handful of labeled examples placed in context lets a pretrained model perform new tasks it was never explicitly fine-tuned for [**?**], and has recently been extended from VLMs to VLAs: prior work trains policies that use a handful of demonstrations as context, boosting in- and out-of-domain performance without any fine-tuning stage. A central open question is what to retrieve – i.e., which stored demonstration segment a policy should select and place into its context window for a given query – and how; existing methods rely on visual similarity, alone or combined with proprioception [4], [5]. We argue that task-relative progress is an important and underexplored complement to this signal.

c) Value function estimators.: Value function estimators (VFEs) have surged in popularity for VLAs, since they provide dense reward signals for RL that push performance past the ceiling of imitation learning alone [2], and general-purpose VFEs built on VLM backbones [6]–[8] are an active area of work. But general-purpose VFEs still gain from task-specific fine-tuning, reintroducing a second training round before deployment. We observe that these transformer-based VFEs can instead benefit from the same ICL machinery already applied

to VLMs and VLAs: to the best of our knowledge, this work proposes the first in-context value function estimator, conditioning on a full-length demonstration to boost value-assignment quality on out-of-domain or long-horizon tasks without task-specific fine-tuning.

d) VICTR.: This work presents VICTR. At test time, VICTR accepts a language query and a small set of teleoperated demonstrations (1–10), and uses an In-Context Value Function Estimator (IC-VFE) to obtain an estimated value $\hat{v}_t$ for the current observation (Section III-A). This estimate is fused with visual similarity, following RICL's retrieval formulation [4], to retrieve a context chunk from the demonstration pool, which then conditions the IC-VLA's next action-chunk prediction. To enable both the IC-VFE and the IC-VLA to use demonstrations effectively, both are meta-trained on the same held-out context pool per task, using oracle value annotations for retrieval during training; at deployment, those oracle values are replaced by the IC-VFE's own estimates. We evaluate the full method on 12 in-domain and 12 novel tasks (Section IV-B) across multiple analysis axes, and separately validate its robustness on the LIBERO-100 benchmark (Section IV-A).

In summary, this work makes three primary contributions:

1) The first In-Context Value Function Estimator, which takes a full-length demonstration as context and provides superior value estimation on in- and out-of-domain tasks relative to current foundation value-estimation approaches.
2) The use of estimated state value as a retrieval signal for in-context VLA learning, fused with visual similarity via a learned scalar combination, extending prior work that relies on visual similarity alone.
3) A rigorous evaluation of VICTR, an ablation of its design elements, and a comparison against state-of-the-art in-context learning methods on a real bimanual manipulator trained on 22.6 hours across 44 tasks of teleoperated data, complemented by a robustness ablation on the LIBERO-100 benchmark.

II. Related Work

a) In-context learning for embodied agents.: Prior work has explored injecting in-context adaptability into VLAs post hoc by retrieving and placing demonstration segments directly in a policy's context window [4], learning single-demonstration ICL for non-VLA policies via next-token prediction [9], and performing in-context imitation learning with explicit visual reasoning [10]. Other work targets meta-ICL from unstructured human play data [5], studies how retrieved action consequences shown in-context help VLAs generalize [11], or replaces sequence-conditioned retrieval with explicit point-correspondence matching between a single demonstration and the current scene [12]. A parallel line of work scales context implicitly rather than by retrieval, compressing long interaction histories into online-updated fast weights via test-time training [13]; VICTR instead keeps policy parameters fixed at test time and relies entirely on what is retrieved into the context window. VICTR builds most directly on the retrieval-augmented formulation of RICL [4], extending its visual-similarity-only retrieval signal with an in-context value estimate.

b) Value and reward modeling for VLAs.: A growing body of work trains general-purpose value/reward models for robot manipulation, including stage-aware reward modeling [8] and other VLM-backboned value estimators [6], [7]. RECAP [2] shows that a distributional value function trained on episode-level success labels can drive advantage-conditioned policy improvement at scale; our IC-VFE adopts a similar distributional value-bin formulation (Section III-C) but conditions the prediction on an in-context demonstration rather than the query observation alone. Closest in spirit is VITA [14], which also targets out-of-domain value calibration but does so by taking online gradient steps on a self-supervised loss at test time; the IC-VFE instead performs this calibration entirely through attention over an in-context demonstration, requiring no test-time parameter updates. Advantage-Weighted Flow Matching [15] similarly uses value annotations to bias flow-matching-based policy training toward high-reward behavior; we use it as a policy ablation (Appendix E) rather than as part of the core method.

c) Retrieval in robot learning.: Beyond ICL specifically, retrieval has been used to augment behavior-cloning policies with relevant offline data [16], and dedicated retrieval-augmented VLA architectures have been proposed for general manipulation [**?**]. LEACL explores LLM-enhanced automatic curriculum construction for robot learning [17]. VICTR differs from this line of work in retrieving based on a learned, task-progress-aware signal rather than similarity alone.

III. Methodology

In this section we present an overview of VICTR, the architecture of our In-Context Value Function Estimator (IC-VFE) and the IC-VLA policy, the construction of our meta-training and evaluation data, and our training objectives.

A. Problem Formulation

We consider a multi-task vision-language-action setting. A task is specified by a language instruction $\ell \in \mathcal{L}$, and an observation $o_t = [X_t^1, \ldots, X_t^n, q_t]$ consists of n camera images and proprioceptive state q_t, following the notation of $\pi_{0.5}/\pi_{0.6}$ [1], [2]. A base VLA policy $\pi_\theta(a_{t:t+H}, \hat{\ell} \mid o_t, \ell)$ maps an observation and instruction to an action chunk $a_{t:t+H}$ and a predicted subtask token $\hat{\ell}$.

a) Demonstration pool (meta-training).: For each meta-training task ℓ, we hold out 10% of its demonstrations as a shared context pool $\mathcal{D}_\ell^{\text{ctx}} \subset \mathcal{D}_\ell$, with the remaining 90%, $\mathcal{D}_\ell^{\text{train}}$, supplying query pairs $(o_t, a_{t:t+H})$ for the base cross-entropy / flow-matching losses. Each demonstration $\tau = (o_0, a_0, \ldots, o_T)$ is annotated offline (Section IV-A) with a per-frame subtask label s_t and a scalar progress value $v_t \in [0, 1]$, with $v_0 \approx 0$ and $v_T = 1$ on success. We downsample each

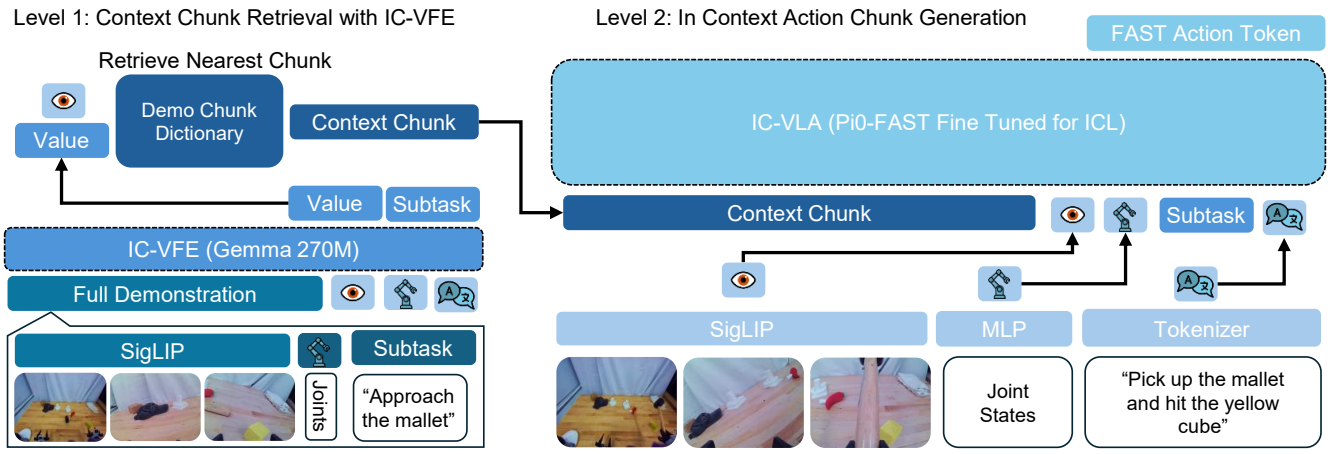


Fig. 1: **VICTR method overview (placeholder).** At deployment, the demonstration pool $\mathcal{D}_\ell$ feeds two in-context estimators. The IC-VFE (ϕ) conditions on one context demonstration to produce a calibrated progress estimate $\hat{v}_t$ for the current observation o_t; $\hat{v}_t$, fused with visual similarity, drives retrieval of a context chunk c_t from the same pool. The IC-VLA (θ) conditions on c_t to predict the next action chunk, closing the loop without any parameter updates. **[GAP: placeholder figure – replace with final diagram once the layout in Section III-B is finalized.]**

demonstration at stride Δ (30 frames at 30 fps, i.e. 1 Hz) into a *context sequence*

$$C(\tau) = \big(b_1, \ldots, b_M\big), \qquad b_j = (o_{j\Delta},\, s_j,\, v_j), \qquad M = \lfloor T/\Delta \rfloor \tag{1}$$

Critically, the same held-out set $\mathcal{D}_\ell^{\text{ctx}}$ feeds both estimators in Figure 1, but in two different forms. First, one whole demonstration $\tau_{\text{ctx}} \sim \mathcal{D}_\ell^{\text{ctx}}$ is sampled and given to the IC-VFE as the context sequence it conditions on to produce $\hat{v}_t$. Second, $\mathcal{D}_\ell^{\text{ctx}}$ is broken apart into individual frames to form a separate frame-level retrieval buffer $\mathcal{P}_\ell$ (defined below), from which the IC-VLA's retrieval step selects the context chunk c_t. In short: one full demonstration primes the IC-VFE, while the same pool, viewed as a flat set of frames, supplies candidates for the IC-VLA's retrieval. This mirrors the "priming vs. retrieval buffer" split in RICL [4], except here the same held-out set serves both roles for both estimators, keeping the two meta-training procedures aligned on identical context data.

b) Demonstration pool (deployment).: At deployment there is no train/context split: the user supplies a single small demonstration set $\mathcal{D}_\ell$ ($N \leq 10$) for the new task, and this entire set — not a 10% subset of it, unlike the meta-training case above — directly plays the role that $\mathcal{D}_\ell^{\text{ctx}}$ played during meta-training. That is, all N demonstrations are simultaneously (a) the candidate pool the IC-VFE's context demonstration is drawn from, and (b) the frame-level retrieval buffer $\mathcal{P}_\ell$ the IC-VLA retrieves from, exactly as in meta-training, just without the held-out split that was only needed there to keep a portion of the data available for the base cross-entropy/flow-matching losses.

c) In-Context Value Function Estimator (IC-VFE).: Rather than estimating value from the query observation alone (as in RECAP's $p_\phi(V \mid o_t, \ell)$ [2]), the IC-VFE additionally conditions on one full context demonstration's bin sequence:

$$p_\phi\big(V \mid o_t, \ell, C(\tau_{\text{ctx}})\big) \in \Delta^B, \tag{2}$$

a distribution over B discretized value bins (following RECAP's distributional value head), from which we recover a scalar estimate $\hat{v}_t = \sum_b p_\phi(V = b \mid \cdot)\, v(b)$. Intuitively, the context demonstration lets the IC-VFE calibrate what progress looks like for this specific task, object, or embodiment instance before scoring the live observation, rather than relying purely on generalization from pretraining.

d) Context-chunk retrieval pool.: We flatten the same N-demonstration pool introduced above — $\mathcal{D}_\ell^{\text{ctx}}$ during meta-training, or $\mathcal{D}_\ell$ at deployment — into a frame-level retrieval buffer

$$\mathcal{P}_\ell = \bigcup_{i=1}^{N} \Big\{ \big(o_\tau,\, a_{\tau:\tau+H},\, v_\tau\big) \Big\}_{\tau=0}^{T^{(i)}}, \tag{3}$$

where each frame carries its oracle value (during meta-training) or IC-VFE-estimated value (at deployment). $\mathcal{P}_\ell$ is what Figure 1 depicts as the demonstration pool feeding retrieval, and what Section III-B formalizes as a chunk-indexed key–value dictionary; here we treat it simply as a flat set of per-frame tuples. A retrieval function

$$f_{\text{retrieve}}\big(o_t, \hat{v}_t, \mathcal{P}_\ell\big) \;\to\; c_t = \big(n^{(1)}, \ldots, n^{(k)}\big) \tag{4}$$

selects the k nearest neighbors $n^{(i)} = (o', a', v')$ from $\mathcal{P}_\ell$, ranked by a fused score over visual similarity $d_{\text{vis}}(o_t, o')$ (e.g. DINO-v2 [18] embedding distance, as in RICL) and value alignment $|\hat{v}_t - v'|$. We defer the concrete form of this fusion — a learned scalar combination $g_\psi\big(d_{\text{vis}}, |\hat{v}_t - v'|\big)$ — to Section III-G.

e) IC-VLA.: The retrieved neighbors are placed in the IC-VLA's context, ordered nearest-to-farthest (following RICL's convention), giving a context-conditioned policy

$$\pi_\theta\big(a_{t:t+H}, \hat{\ell} \mid o_t, \ell, c_t\big). \tag{5}$$

f) Bi-level structure.: We call the overall method VICTR: two independent in-context estimators compose at inference time: (1) the IC-VFE conditions on a context demonstration $\tau_{\text{ctx}} \sim \mathcal{D}_\ell^{\text{ctx}}$ to produce $\hat{v}_t \in [0,1]$, and (2) that estimate, jointly with visual similarity, conditions a second retrieval process that selects the context chunk c_t consumed by the IC-VLA. Both the IC-VFE (ϕ) and the IC-VLA (θ) are meta-trained on the same held-out context pool $\mathcal{D}_\ell^{\text{ctx}}$ per task, using oracle progress annotations $v_t \in [0,1]$; at deployment, oracle values are replaced by $\hat{v}_t$ from the trained IC-VFE.

B. Demonstration Chunk Dictionary

The retrieval pool $\mathcal{P}_\ell$ introduced above (Equation 3) is built and indexed as a *demonstration chunk dictionary*: a key–value store whose keys are DINO [18] embeddings of a chunk's first frame, and whose values are the chunk contents themselves. Each stored chunk value contains, for every frame in the chunk, the top-camera image tokens, left-wrist image tokens, right-wrist image tokens, subtask tokens, proprioception tokens, and action tokens.

To build the dictionary for a given task, we start from k demonstrations of that task and split each into overlapping chunks of a fixed chunk size L (in frames), with 50% overlap between consecutive chunks. Concretely, a demonstration of T frames contributes

$$\left(\frac{T}{L} \times 2\right) - 1 \tag{6}$$

chunks to the dictionary. The chunk dictionary built from a task's k demonstrations is exactly the structure that f_{retrieve} (Equation 4) queries against at both meta-training and deployment time.

a) Context window token layout.: At both meta-training and deployment time, f_{retrieve} returns the k neighbor chunks $c_t = (n^{(1)}, \ldots, n^{(k)})$ already defined in Equation 4 (distinct from the k demonstrations used above to build the chunk dictionary itself). Each neighbor chunk contributes one token block per frame, reusing the same per-frame tokens defined above for the chunk dictionary, concatenated in frame order within a neighbor; the k neighbor blocks are then concatenated in nearest-to-farthest order (as ranked by f_{retrieve}) ahead of the query observation tokens. Each neighbor block additionally receives a learned neighbor-rank embedding (added to every token in the block) so the IC-VLA can distinguish nearest-neighbor context from farther, lower-ranked context.

C. IC-VFE Architecture and Demonstration Bin Tokenization

Given a language instruction ℓ, the current observation $o_t = [X_t^1, \ldots, X_t^n, q_t]$ (reusing the notation of Section III-A; here $n = 3$ cameras), and a successful context demonstration $\tau_{\text{ctx}} = (o_0, \ldots, o_{T_{\tau_{\text{ctx}}}-1})$ of the same task (also Section III-A), IC-VFE predicts the relative task-completion value of o_t. IC-VFE conditions its prediction on a temporally compressed representation of τ_{ctx}, a high-resolution representation of the current observation o_t, the task instruction ℓ, and a learnable value-query token. A no-context ablation removes only the demonstration sequence while retaining the same current-observation encoder and prediction head.

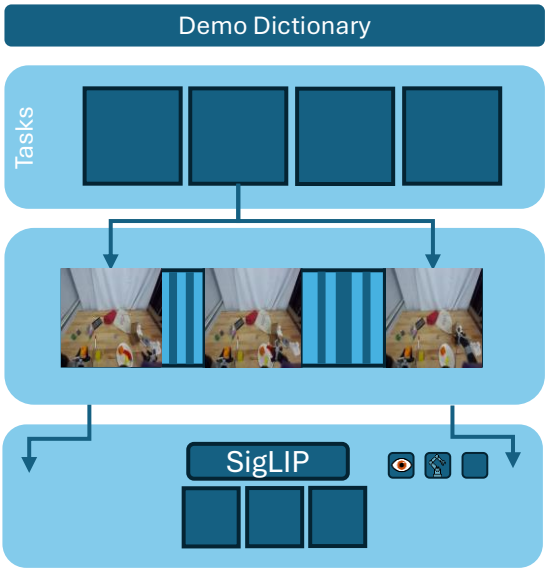


Fig. 2: **Demonstration chunk dictionary construction (placeholder).** Illustration of splitting a demonstration into overlapping chunks of size L and indexing each by the DINO embedding of its first frame. **[GAP: placeholder figure – replace with final diagram.]**

a) Architecture summary.: The context demonstration is downsampled end-anchored (most recent frames preferred) into a fixed number of temporal *context slots*, each visually tokenized by a frozen Gemma 3 vision tower and trainable patch merger, alongside a proprioceptive token; the current observation is separately encoded at two resolutions so the query frame shares a common visual space with the demonstration. Context tokens, current-observation tokens, proprioceptive state, the tokenized instruction, and a learnable value-query token are concatenated and processed by a Gemma 3 270M causal decoder; the value-query token's final hidden state is mapped by a value head to a categorical distribution over 201 discretized progress bins. Exact context-slot counts, token budgets, and the pooling operator are given in Appendix C.

b) Demonstration-derived value-bin tokenization.: The value target is defined by the relative progress of the current query frame within its successful demonstration. For a successful trajectory $\tau = (o_0, \ldots, o_{T-1})$, the continuous value assigned to frame t is

$$y_t = v_{\min} + \frac{t}{T-1}(v_{\max} - v_{\min}), \qquad T > 1, \tag{7}$$

with $v_{\min} = -1$ and $v_{\max} = 0$; the initial frame has value -1, the terminal successful frame has value 0, and intermediate frames are labeled by linear relative progress. For the degenerate case $T = 1$, the single terminal frame

is assigned $y_0 = v_{\max} = 0$. We discretize this continuous target into $B = 201$ uniformly spaced value bins. The scalar represented by bin $b \in \{0, \ldots, B-1\}$ is

$$\nu_b = v_{\min} + \frac{b}{B-1}(v_{\max} - v_{\min}), \tag{8}$$

so adjacent bins are separated by $1/200 = 0.005$. The target bin index for value y_t is

$$b_t = \text{clip}\left(\text{round}\left[\frac{y_t - v_{\min}}{v_{\max} - v_{\min}}(B-1)\right], 0, B-1\right). \tag{9}$$

We refer to this operation as demonstration-derived value-bin tokenization because the categorical target is obtained from the query frame's position in a successful demonstration. These 201 value bins define the output vocabulary of the value head; they are supervision classes rather than tokens appended to the multimodal input sequence. The 201 value bins should not be confused with the 200 temporal context slots (Appendix C, Table IV).

D. IC-VFE Meta-Learning Training Objective

Let $z_\theta(o_t, \ell, \tau_{\text{ctx}}) \in \mathbb{R}^B$ denote the logits produced by the value head, where $B = 201$. The predicted categorical value distribution is

$$p_\theta(b \mid o_t, \ell, \tau_{\text{ctx}}) = \frac{\exp z_{\theta,b}}{\sum_{b'=0}^{B-1} \exp z_{\theta,b'}}. \tag{10}$$

IC-VFE is trained with categorical cross-entropy against the demonstration-derived target bin b_t,

$$\mathcal{L}_{\text{IC-VFE}}(\theta) = -\mathbb{E}_{(o_t,\ell,\tau_{\text{ctx}})\sim\mathcal{B}}\left[\log p_\theta(b_t \mid o_t, \ell, \tau_{\text{ctx}})\right], \tag{11}$$

where $\mathcal{B}$ denotes the training distribution over query frames, instructions, and same-task context demonstrations sampled from $\mathcal{D}_\ell^{\text{ctx}}$ (Section III-A). For the no-context ablation, the objective is unchanged and the conditioning variable τ_{ctx} is omitted.

The frozen vision tower and fixed average-pooling operator are excluded from optimization. Gradients update the detail connector, shared patch merger, proprio encoder, learned source embeddings, Gemma 270M backbone, value-query token, and value head. In the context model, the shared merger receives gradients through both the cached demonstration features and the online current observation; in the no-context model, it receives gradients through the current observation only.

At inference time, a scalar value is recovered as the expectation of the predicted categorical distribution,

$$\hat{y}_t = \mathbb{E}_{p_\theta}[\nu_b] = \sum_{b=0}^{B-1} p_\theta(b \mid o_t, \ell, \tau_{\text{ctx}})\, \nu_b. \tag{12}$$

This distributional formulation preserves uncertainty over progress while providing a scalar estimate in $[-1, 0]$ when required. Mean-squared error and mean-absolute error between $\hat{y}_t$ and y_t may be reported as diagnostics, but they are not auxiliary optimization terms: the training loss is the categorical cross-entropy alone.

E. IC-VLA Architecture and Context Chunk Tokenization

The IC-VLA shares its base architecture and fine-tuning regime with $\pi_{0.5}$ (Table V), extended to accept the retrieved context chunk c_t (Section III-A, Equation 4) alongside the query observation, following the general recipe of placing retrieved neighbors in context ordered nearest-to-farthest (as in RICL [4]). The token-level layout of this context window — how each retrieved neighbor is tokenized, how many neighbors are retrieved, and how neighbor tokens are ordered relative to the query — is given in Section III-B.

F. IC-VLA Meta-Learning Training Objective

Following $\pi_{0.5}$'s factorized log-likelihood over a discrete subtask prediction and a continuous action chunk, the IC-VLA is trained with a combined objective over the context-conditioned policy $\pi_\theta(a_{t:t+H}, \hat{\ell} \mid o_t, \ell, c_t)$ (Equation 5). The subtask head produces a categorical distribution $p_\theta(\hat{\ell} \mid o_t, \ell, c_t)$ over subtask tokens, trained with autoregressive cross-entropy against the ground-truth subtask label s_t; the action expert predicts the continuous action chunk $a_{t:t+H}$ conditioned on the same context, trained with the continuous flow-matching loss L_{flow}, following the fine-tuning regime in Table V (Appendix D). The combined objective is

$$\mathcal{L}_{\text{IC-VLA}}(\theta) = \mathbb{E}_{(o_t,\ell,c_t)\sim\mathcal{B}}\Big[-\log p_\theta(s_t \mid o_t, \ell, c_t) + L_{\text{flow}}\big(a_{t:t+H} \mid o_t, \ell, c_t; \theta\big)\Big], \tag{13}$$

mirroring the stop-gradient interface between the VLM backbone and the action expert used for the base $\pi_{0.5}$ fine-tune (Table V). During meta-training, the context chunk c_t is sampled by f_{retrieve} (Equation 4) using oracle progress values v_t from $\mathcal{D}_\ell^{\text{ctx}}$ (Section III-A); at deployment, oracle values are replaced by the trained IC-VFE's estimate $\hat{v}_t$, so retrieval quality at inference time is bounded by IC-VFE accuracy. **[GAP: confirm accuracy.]**

G. Vision and Value aware Context Chunk Retrieval

The retrieval function f_{retrieve} introduced in Section III-A ranks candidate neighbors by a fused score over visual similarity $d_{\text{vis}}(o_t, o')$ (DINO-v2 [18] embedding ℓ_2 distance, as in RICL) and value alignment $|\hat{v}_t - v'|$. We adopt a *learned scalar fusion* $g_\psi(d_{\text{vis}}, |\hat{v}_t - v'|)$ rather than a fixed weighting (e.g. RICL's fixed distance-decay $e^{-\lambda d}$) or a two-stage filter, so that the relative importance of visual and value similarity can be learned rather than hand-tuned. Concretely, g_ψ is a small 2-layer MLP that takes the 2-dimensional input $[d_{\text{vis}}(o_t, o'), |\hat{v}_t - v'|]$ and outputs a single fused ranking score, used to select the top-k neighbors for c_t. We train g_ψ jointly with the IC-VLA: since top-k selection is not itself differentiable, g_ψ's parameters receive gradient signal indirectly through the IC-VLA's training objective (Equation 13) by scoring the full retrieval-pool candidate set with a soft (temperature-annealed softmax) relaxation of top-k selection during meta-training, and switching to hard top-k selection at evaluation and deployment time.

IV. Experimental Setup

We evaluate VICTR along three axes: (1) design ablations for the IC-VFE in isolation, (2) design ablations for the IC-VLA in isolation, and (3) a main evaluation comparing the full method against retrieval-based and in-context-learning baselines on a held-out set of in-domain and out-of-domain tasks. We additionally run a targeted ablation on the number of demonstrations available at deployment time, to characterize the few-shot regime the method is designed for. As a robustness check on a common simulated benchmark, we separately evaluate VICTR on LIBERO-100 (Section IV-A).

A. Dataset and Task Suite

Our meta-training dataset consists of teleoperated bimanual manipulation data spanning pick-and-place, insertion, bimanual assembly, articulated-object manipulation (e.g. uncapping, unscrewing), and long-horizon composite tasks across 44 tasks. In total the corpus comprises 3,162 episodes (2,717 successful, 337 failed) totaling 22.6 hours at 30 fps across three cameras; full per-task episode/success/fail counts and file-level statistics are reported in Appendix A. Episodes are auto-labeled with subtask/progress annotations via an offline VLM-based pipeline, and training episodes are augmented with a mirrored (left/right-flipped) copy to reduce bimanual arm bias; full annotation and augmentation details, including the fine-tuning regime, are given in Appendix D. We additionally evaluate all comparisons on LIBERO-100 (LIBERO-90 + LIBERO-Long) [19], using an analogous 80/20 task split and demonstration-pool construction; policy-side augmentations specific to bimanual manipulation (MSA, AWFM, the baseline-architecture sweep) are not evaluated there. Given compute/time limits, we run $k = 3$ seeded repetitions of the main comparisons on LIBERO-100 only.

B. Evaluation Task Suite and Demonstration Pools

To evaluate generalization, we construct 12 indexed in-domain/out-of-domain task pairs, where the out-of-domain task shares a manipulation skill with its in-domain counterpart but varies the target object, distractor set, or task composition (e.g. "3 Cup Stack" vs. "3 Cube Stack," "Circle on Peg" vs. "Square on Peg"); the full pairing list is given in Table III (Appendix A). These 12 indices are the canonical numbering used throughout the paper (e.g. in Section V) whenever we refer to "task i." For each of the resulting 24 evaluation tasks we collect a demonstration pool that serves as the IC-VFE's context source, the IC-VLA's retrieval buffer $\mathcal{D}_\ell$ (Section III-A), and the held-out evaluation set for the VFE accuracy metric in Section IV-C; full pool construction, size, and the in-domain/out-of-domain task-index breakdown are given in Appendix A.

C. Value Function Estimator Ablations

We evaluate IC-VFE design choices by measuring (a) training error on the in-domain meta-training data, and (b) held-out VFE accuracy on the 240 evaluation demonstrations from Section IV-B (120 in-domain, 120 out-of-domain). VFE accuracy is measured as error between the predicted value estimate and a human-annotated ground-truth value for each frame.

a) A — Vision encoding.: We compare native visual tokenization at stride 10, native tokenization at stride 20, and patch-merger tokenization at stride 10.

b) B — Data annotation methods (using the best configuration from A).: We compare three sources of value/subtask supervision: the SARM annotation step; sparse reward; and manual annotation combined with subtask prediction. Manual annotation alone is excluded as a compared condition, since it is also the ground truth all conditions are scored against (Appendix B).

c) C — Value function estimator comparison.: We compare our best configuration from Ablations A–B (i.e. our proposed IC-VFE) against RECAP fine-tuned on our data, Robometer, Robometer fine-tuned, and TOPReward.

The policy-side design choices are separately ablated along three axes — baseline architecture, Advantage-Weighted Flow Matching (AWFM), and Morphological Symmetry Augmentation (MSA) — each evaluated with 10 rollouts per task on the 12 in-domain tasks; see Appendix E for the full protocol and Section V-B for results.

D. Main Evaluation

We evaluate all methods on the full set of 24 tasks from Table III, with 10 rollouts per task per method. Each context-retrieval method is instantiated using the best configuration identified in the ablations of Section IV-C and Appendix E (e.g. RICL may use $\pi_{0.5}$ with MSA and AWFM as its backbone); this instantiation is pending completion of those ablations. We compare:

- $\pi_{0.5}$ (no in-context retrieval)
- RICL [4]
- ICRT (single-task policy) [9]
- ICLR (visual-reasoning in-context imitation learning) [10]
- Ours (value-based chunk retrieval)
- Ours (value + vision-similarity retrieval)

Evaluations are conducted on the YOR bimanual manipulator [**?**]; full metrics definitions, robot configuration, judging protocol, and compute budget are given in Appendix H.

As a follow-on once the real-world evaluation is complete, we plan to replicate these comparisons in the CALVIN simulated benchmark, complementing the LIBERO-100 robustness ablation already reported in Section V-C.

E. Demonstration-Count Ablation

To characterize how retrieval-buffer size affects task success, we evaluate VICTR on 6 long-horizon tasks — 3 in-domain (3 Cup Stack, Colored Octagon Stack, Long Horizon Sorting) and their 3 out-of-domain counterparts (Stack 3 Cubes, Stack 2 Cubes in Box and Hit with Mallet, Sort Food into 3 Cups), corresponding to task indices 1, 3, and 6 in Table III — with $\{1, 3, 5, 7, 10\}$ demonstrations provided in context, randomly selected from the available pool, for 10

rollouts per configuration ($6 \times 10 \times 5 = 300$ real-world rollouts total).

V. Results and Analysis

[GAP: this entire section reports placeholder/expected-format results. All numeric values below are illustrative (marked with X.X or TBD) pending real rollouts from Sections IV-C–IV-E, and pending LIBERO-100 rollouts (Section IV-A).]

A. Value Function Estimator Ablations

Table VI (Appendix F) reports value-estimation error for each IC-VFE design choice from Section IV-C, evaluated on the 240 held-out demonstrations (120 in-domain, 120 out-of-domain) described in Section IV-B; the same table also reports the analogous LIBERO-100 VFE comparison (Section IV-A). Every condition is scored by MAE against manually annotated progress values, which we treat as ground truth (Appendix B); manual annotation is therefore excluded as a compared condition in Ablation B, since it would trivially score as its own ground truth.

B. Policy Ablations

Table VII (Appendix F) reports in-domain success rate for each policy ablation from Appendix E, over 10 rollouts per task on the 12 in-domain tasks.

C. Main Evaluation

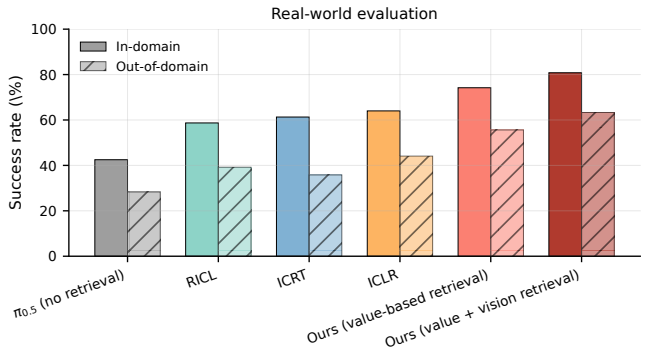
Figure 3 summarizes overall task success across the 24-task evaluation suite (Section IV-D), split by in-domain and out-of-domain task, and further restricted to the same-skill/novel-object task indices (1, 2, 4, 5, 8, 9, 10, 11, 12) for the in-domain-skill-vs-out-of-domain-object comparison described in Appendix H. The same figure additionally reports the LIBERO-100 robustness comparison (Section IV-A), stacked below the real-world results rather than as a separate figure.

D. Effect of Number of Context Demonstrations

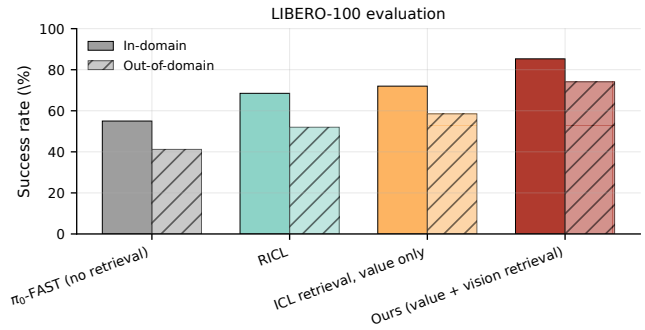
Figure 4 shows success rate as a function of the number of context demonstrations, from the ablation in Section IV-E (6 tasks, $\{1, 3, 5, 7, 10\}$ demonstrations, 10 rollouts per configuration).

E. Recovery Success Rate

Table I reports the recovery success rate metric defined in Appendix H: the fraction of rollouts in which the policy fails an intermediate subtask but recovers to complete it, and the fraction of those recoveries that go on to complete the full task.



(a) Real-world evaluation



(b) LIBERO-100 evaluation

Fig. 3: **Total success rate by method (placeholder).** Bars show in-domain and out-of-domain success rate (%) for each compared method, over 10 rollouts per task per method, on the real-world suite **(a)** (24 tasks, Section IV-D) and on LIBERO-100 **(b)** (Section IV-A); error bars TBD pending the statistical protocol in Appendix H. **[GAP: placeholder figure – replace with real results once Section IV-D rollouts are complete.]**

TABLE I: **Recovery success rate (placeholder).** "Subtask recovery" is the fraction of subtask failures the policy recovers from; "full-task recovery" is the fraction of those recoveries that go on to complete the entire task.

Method	Subtask recovery (%)	Full-task recovery (%)
$\pi_{0.5}$ (no retrieval)	XX.X	XX.X
RICL	XX.X	XX.X
Ours (value + vision retrieval)	**XX.X**	**XX.X**

VI. Limitations and Future Work

VICTR's evaluation protocol has several known limitations. Success and failure are judged against a fixed per-task rubric without inter-rater agreement statistics, and no multiple-seed reruns are performed per configuration (Appendix H). Composition and confounding generalization are currently evaluated only on the 3 long-horizon and 9 same-skill task pairs in Table III, respectively; broader coverage would strengthen the generalization claims in Section I. The LIBERO-100 robustness ablation (Section IV-A) reuses only the VFE and policy comparisons, not the full real-world ablation battery (vision encoding, annotation method, AWFM, MSA), since several axes are specific to the bimanual dataset.

The bi-level design also introduces an unmeasured compounding-error risk: an inaccurate IC-VFE estimate $\hat{v}_t$ can

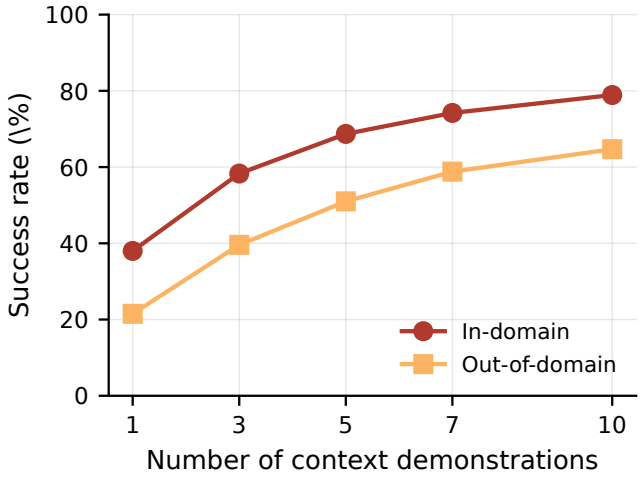


Fig. 4: **Success rate vs. number of context demonstrations (placeholder).** One line per method (or per in-domain/out-of-domain split); x-axis $\in \{1, 3, 5, 7, 10\}$ demonstrations, y-axis success rate (%), pooled over the 6 tasks in Section IV-E. **[GAP: placeholder figure – replace with real results.]**

steer retrieval toward a mismatched context chunk, degrading policy performance independent of the policy's own quality. Section V-A reports IC-VFE accuracy in isolation; an ablation measuring downstream policy sensitivity to VFE error would clarify this failure mode.

Future work includes supervised fine-tuning on top of RL100-style data collection, extending VICTR's context beyond a single retrieved chunk toward longer-horizon composed retrieval, and replicating the LIBERO-100 ablation on CALVIN as a further common-benchmark check (Section IV-D). VICTR is currently training-free at test time by design, relying entirely on retrieval into a fixed-parameter context window; recent theoretical and empirical work shows that pairing in-context learning with lightweight test-time training can further reduce the number of demonstrations needed to reach a target accuracy, both in general transformer ICL settings [20], [21] and, concretely, in VLA policies via fast-weight context scaling [13] and test-time-trained visual foresight [22]. A natural extension of VICTR is to let the retrieved context also drive a small number of test-time gradient steps on the IC-VFE or the policy itself, rather than treating retrieval and inference as fully separate stages.

VII. Conclusion

[GAP: write after Results and Analysis are finalized with real numbers.] This work presents VICTR, a bi-level in-context learning method that pairs an In-Context Value Function Estimator with value- and vision-aware retrieval to adapt a pretrained VLA to new tasks from as few as one to ten demonstrations, without task-specific fine-tuning. We evaluate VICTR against RICL, ICRT, and ICLR baselines on a real bimanual manipulator across in-domain, novel-object, and long-horizon composition tasks, with a robustness ablation on LIBERO-100.

Acknowledgment

[GAP: fill in acknowledgments once finalized.]

References

[1] P. Intelligence, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," 2025. [Online]. Available: https://arxiv.org/abs/2504.16054

[2] ——, "$\pi^*_{0.6}$: a vla that learns from experience," 2025. [Online]. Available: https://arxiv.org/abs/2511.14759

[3] P. Atreya *et al.*, "Roboarena: Distributed real-world evaluation of generalist robot policies," 2025. [Online]. Available: https://arxiv.org/abs/2506.18123

[4] K. Sridhar, S. Dutta, D. Jayaraman, and I. Lee, "Ricl: Adding in-context adaptability to pre-trained vision-language-action models," 2025. [Online]. Available: https://arxiv.org/abs/2508.02062

[5] R. S. et al., "Mimicdroid: In-context learning for humanoid robot manipulation from human play videos," 2025. [Online]. Available: https://arxiv.org/abs/2509.09769

[6] A. L. et al., "Robometer: Scaling general-purpose robotic reward models via trajectory comparisons," 2026. [Online]. Available: https://arxiv.org/abs/2603.02115

[7] S. C. et al., "Topreward: Token probabilities as hidden zero-shot rewards for robotics," 2026. [Online]. Available: https://arxiv.org/abs/2602.19313

[8] Q. Chen, J. Yu, M. Schwager, P. Abbeel, Y. Shentu, and P. Wu, "Sarm: Stage-aware reward modeling for long horizon robot manipulation," 2026. [Online]. Available: https://arxiv.org/abs/2509.25358

[9] L. Fu, H. Huang, G. Datta, L. Y. Chen, W. C.-H. Panitch, F. Liu, H. Li, and K. Goldberg, "In-context imitation learning via next-token prediction," 2024. [Online]. Available: https://arxiv.org/abs/2408.15980

[10] T. Nguyen, W. Yuan, S. Wei, H. Li, D. Seita, and Y. Wang, "Iclr: In-context imitation learning with visual reasoning," 2026. [Online]. Available: https://arxiv.org/abs/2603.07530

[11] Q. Lian, K. Yu, and L. Zhang, "Reflective vla: In-context action consequences make vlas generalize," 2026. [Online]. Available: https://arxiv.org/abs/2606.25215

[12] S. Liu *et al.*, "Matchingpolicy: Correspondence-aware policy for cross-object in-context learning," 2025. [Online]. Available: https://openreview.net/forum?id=ddxq6ACoY3

[13] NVIDIA GEAR Lab *et al.*, "Robottt: Context scaling for robot policies," 2026. [Online]. Available: https://arxiv.org/abs/2607.15275

[14] C. Ziakas *et al.*, "Vita: Zero-shot value functions via test-time adaptation of vision–language models," 2025. [Online]. Available: https://arxiv.org/abs/2506.10085

[15] S. Xue, C. Ge, S. Zhang, Y. Li, and Z.-M. Ma, "Advantage weighted matching: Aligning rl with pretraining in diffusion models," 2025. [Online]. Available: https://arxiv.org/abs/2509.25050

[16] A. M. et al., "Mimicgen: A data generation system for scalable robot learning using human demonstrations," 2023. [Online]. Available: https://arxiv.org/abs/2310.17596

[17] F. Heravi, J. Ouyang, Z. Xu, A. Kumar, Y. Sung, and P. Stone, "Leacl: Llm-enhanced automatic curriculum learning for reinforcement learning in long-horizon manipulation tasks," 2026. [Online]. Available: https://arxiv.org/abs/2607.23515

[18] M. O. et al., "Dinov2: Learning robust visual features without supervision," 2024. [Online]. Available: https://arxiv.org/abs/2304.07193

[19] B. L. et al., "Libero: Benchmarking knowledge transfer for lifelong robot learning," 2023. [Online]. Available: https://arxiv.org/abs/2306.03310

[20] H. A. Gözeten *et al.*, "Test-time training provably improves transformers as in-context learners," in *International Conference on Machine Learning (ICML)*, 2025. [Online]. Available: https://arxiv.org/abs/2503.11842

[21] E. Akyürek *et al.*, "The surprising effectiveness of test-time training for few-shot learning," 2025. [Online]. Available: https://ekinakyurek.github.io/papers/ttt.pdf

[22] Anonymous, "Test-time training for visual foresight vision-language-action models," 2026. [Online]. Available: https://arxiv.org/abs/2605.08215

[23] Y. L. et al., "Gr-rl: Going dexterous and precise for long-horizon robotic manipulation," 2025. [Online]. Available: https://arxiv.org/abs/2512.01801

TABLE II: Dataset statistics for the meta-training corpus.

Metric	Value
Episodes	3,162
Frames	2,444,766
Duration	22.6 hours (1,358 min) @ 30 fps
Tasks	44
Cameras	3 (zed, fish0, fish1)
Video files	9,486
Parquet files	3,162 (32 chunks of 100)
Total files on Hub	12,654
Size	∼15.5 GB
Source runs merged	53

TABLE III: Indexed in-domain / out-of-domain evaluation task pairs. These indices are referenced directly in later figures and analysis.

#	In-domain task	Out-of-domain task
1	3 Cup Stack	3 Cube Stack
2	Circle on Peg	Square on Peg
3	Colored Octagon Stack	Stack 2 cubes in box and hit with mallet
4	Bag bell pepper	Bag chili
5	Hit yellow cube	Hit eggplant
6	Long horizon sorting	Sort foods into 3 cups
7	Open notebook	Open notebook and place cube on page
8	Uncap gatorade	Uncap bottle
9	Pass shaker	Pass mustard to plate
10	PnP red chili to plate	PnP gusset to plate
11	Uncap red marker	Uncap black marker
12	PnP eggplant box	PnP mustard box

APPENDIX

A. Dataset and Task Suite Details

Table II reports full dataset statistics for the meta-training corpus (Section IV-A), and Table III lists the 12 indexed in-domain/out-of-domain evaluation task pairs (Section IV-B).

Task pairs 3, 6, and 7 are long-horizon compositions of previously seen sub-skills rather than same-skill/novel-object substitutions; the remaining nine pairs (1, 2, 4, 5, 8, 9, 10, 11, 12) isolate a single skill against a novel object or a visually confounding "cousin" object under an otherwise fixed skill. Per-task episode/success/fail counts (2,717 successful and 337 failed episodes out of 3,162 total, in aggregate) are reported in Table II.

For each of the 24 evaluation tasks (Section IV-B) we collect approximately 10 successful episodes plus a variable number of failure episodes, giving ∼240 demonstrations total, split evenly as 120 in-domain and 120 out-of-domain. This demonstration set serves three roles across the paper: it is the IC-VFE's context source and the IC-VLA's retrieval buffer $\mathcal{D}_\ell$ at deployment (Section III-A), and it is also the held-out evaluation set for the VFE accuracy metric in Section IV-C. All baseline methods (RICL, ICRT, ICLR) are retrained from scratch on our meta-training dataset rather than evaluated from released checkpoints, so that all compared methods see the same underlying data.

B. Human Annotation / Ground-Truth Protocol

All VFE comparisons (Table VI, Appendix F) are scored by mean absolute error against a manually annotated progress value for each frame, which we treat as ground truth, distinct from the task-success rubric used for policy rollouts (Appendix H). Raters view each demonstration alongside its subtask segmentation (Appendix D) and assign a progress value in $[0, 1]$ at each subtask boundary, which is then linearly interpolated within each subtask to produce a dense per-frame label. **[GAP: fully specify the rater pool, inter-rater agreement protocol, and the exact interpolation rule once finalized.]**

C. IC-VFE Implementation Details

This appendix gives the exact context-slot sampling procedure, frozen-vision tokenization pipeline, and token budgets underlying the IC-VFE architecture summarized in Section III-C.

a) Demonstration sampling and context slots.: We retain at most the final $W = 6000$ timesteps of the context demonstration and sample them with stride $s = 30$, giving a maximum number of context slots

$$M = \left\lceil \frac{W}{s} \right\rceil = 200. \tag{14}$$

For a demonstration of length $T_{\mathcal{D}}$, let

$$K = \min\left(M,\ \left\lceil \frac{\min(T_{\mathcal{D}}, W)}{s} \right\rceil\right) \tag{15}$$

be the number of valid sampled timesteps. Following an end-anchored sampling rule, frames are first selected backward from the terminal frame,

$$\mathcal{J}_{\text{rev}} = \{T_{\mathcal{D}} - 1 - ks\}_{k=0}^{K-1}, \tag{16}$$

and then reordered chronologically. When $K < M$, the valid samples are right-aligned and the first $M - K$ slots are padding, which is removed by the context attention mask. The resulting 200 positions are *context slots*, distinct from the 201 value bins defined in Section III-C.

b) Frozen native-vision representation.: We use the native Gemma 3 vision tower to encode all context and current RGB images. Let ϕ_{vis} denote the frozen vision tower and $A_{4\times4}$ the fixed spatial average-pooling operator. For camera c at timestep j, the pooled visual representation is

$$X_j^c = A_{4\times4}\big(\phi_{\text{vis}}(I_j^c)\big) \in \mathbb{R}^{256\times1152}. \tag{17}$$

The vision tower runs in evaluation mode and receives no gradients. Average pooling is parameter-free, and the pooled representation is detached before entering any trainable visual adapter. For context demonstrations, X_j^c is computed offline and cached in bf16; a context shard with K valid slots and three cameras has shape $K \times 3 \times 256 \times 1152$. This cache is produced before visual projection or patch merging, allowing the trainable context adapter to be optimized without repeatedly evaluating the frozen vision tower.

c) Context tokenization.: A trainable spatial patch merger g_θ compresses each cached visual grid from 256 tokens to 16 tokens and projects the channel dimension from 1152 to the 640-dimensional Gemma 270M embedding space,

$$g_\theta : \mathbb{R}^{256\times1152} \to \mathbb{R}^{16\times640}. \tag{18}$$

A shared proprioceptive encoder f_{prop} maps the robot state to one 640-dimensional token, $f_{\text{prop}}(p_j) \in \mathbb{R}^{1\times640}$. The token block for context slot m is

$$Z_m^{\text{ctx}} = \text{Concat}\left[g_\theta(X_m^1),\, g_\theta(X_m^2),\, g_\theta(X_m^3),\, f_{\text{prop}}(p_m)\right], \tag{19}$$

which contains $3 \times 16 + 1 = 49$ tokens. Learned segment, modality, camera, and temporal-slot embeddings are added to the corresponding tokens. After packing all $M = 200$ slots, the maximum context sequence length is $N_{\text{ctx}} = 200\times49 = 9800$. All 49 tokens belonging to a padded slot share a zero attention-mask value, preserving a fixed tensor layout while preventing padded context from influencing the prediction.

d) Dual-resolution current-observation encoding.: The current observation uses the same frozen native-vision representation, $X_t^c = A_{4\times4}(\phi_{\text{vis}}(I_t^c)) \in \mathbb{R}^{256\times1152}$, but retains two complementary resolutions. First, a trainable detail connector h_θ independently projects every pooled token,

$$h_\theta : \mathbb{R}^{256\times1152} \to \mathbb{R}^{256\times640}. \tag{20}$$

Second, the current image is passed through the same patch merger g_θ used by the context path, $g_\theta(X_t^c) \in \mathbb{R}^{16\times640}$. The 256-token branch preserves fine visual detail, while the shared 16-token branch places the current image in the same compressed visual representation as the demonstration. The current visual block is

$$Z_t^{\text{vis}} = \text{Concat}_{c=1}^{3}\left[h_\theta(X_t^c),\, g_\theta(X_t^c)\right], \tag{21}$$

containing $N_{\text{current-vis}} = 3(256 + 16) = 816$ tokens. The current proprioceptive state contributes one additional token, $f_{\text{prop}}(p_t)$.

e) Multimodal sequence construction.: The instruction is tokenized by the Gemma tokenizer into at most 32 token positions. Let $Z^{\text{text}} \in \mathbb{R}^{32\times640}$ denote the padded instruction embeddings and let $q_V \in \mathbb{R}^{1\times640}$ be a learnable value-query token. IC-VFE constructs the Gemma input sequence as

$$Z = \text{Concat}\left[Z^{\text{ctx}},\, Z_t^{\text{vis}},\, f_{\text{prop}}(p_t),\, Z^{\text{text}},\, q_V\right]. \tag{22}$$

The maximum input length of the context model is $9800 + 816+1+32+1 = 10650$. The no-context ablation omits Z^{ctx} but otherwise uses the same architecture, giving a maximum length of $816 + 1 + 32 + 1 = 850$. The sequence is processed by a Gemma 3 270M causal decoder F_θ. Because q_V is the final token, its hidden state can attend to all preceding context, current-observation, and language tokens,

$$h_V = F_\theta(Z)_{[-1]} \in \mathbb{R}^{640}. \tag{23}$$

A value head maps h_V to a categorical distribution over task progress.

Only the 201 value bins are referred to as bins; the 200 temporal positions are context slots.

TABLE IV: IC-VFE terminology summary, to avoid conflating temporal context layout with value supervision.

Term	Count	Role
Context slots	200	End-anchored, stride-30 temporal samples from the c
Tokens per context slot	49	Three cameras with 16 visual tokens each, plus one p
Context tokens	9,800	Maximum token sequence contributed by all context
Value bins	201	Uniform categorical output classes over relative progr

D. Dataset Preparation, Augmentation, and Fine-Tuning Details

This appendix gives the full subtask/value annotation pipeline, the four-step morphological symmetry augmentation (MSA) transformation, the action representation, and the fine-tuning regime summarized in Section IV-A.

a) Subtask annotation.: $\pi_{0.5}$ is trained to predict subtasks, so long-horizon episodes in our corpus are auto-labeled with subtask segments using a lightweight, offline three-step pipeline. First, kinematic change-point proposals are generated by detecting gripper state changes (open/close transitions) and end-effector velocity minima in the raw trajectory. Second, a contact sheet is rendered by sampling frames at each proposal with timestamp burn-ins. Third, a VLM (e.g. Gemini or GPT-4o) is prompted on the contact sheet to return structured subtask segments as JSON. This pipeline is also used to produce higher-quality progress annotations for value function estimation than would be available from episode-level success labels alone, since it identifies the sub-goal boundaries needed to calibrate within-episode progress. **[GAP: finalize which annotation configuration from Section IV-C, Ablation B, is used as the production subtask/value labeling pipeline.]**

b) Morphological symmetry augmentation (flipping).: To reduce left/right arm bias in our bimanual dataset, we augment training episodes with a mirrored copy across the central vertical axis, requiring four coordinated transformations so that the mirrored episode remains physically and linguistically consistent. The effect of this augmentation (MSA) on policy success rate is measured in Policy Ablation C (Appendix E).

1) *Visual transforms.* Overhead/workspace cameras receive a standard horizontal flip. Wrist cameras must additionally swap the left/right camera channels before flipping, since the right arm's view reflected across the sagittal plane becomes the left arm's view:

$$\text{Wrist}_{\text{left,new}} = \text{Flip}_{\text{h}}(\text{Wrist}_{\text{right,old}}), \quad \text{Wrist}_{\text{right,new}} = \text{Flip}_{\text{h}}(\text{Wrist}_{\text{left,} \tag{24}$$

Flipping a wrist image without swapping its channel first produces impossible perspective geometry.

2) *Kinematic and state swapping.* With Y as the lateral left–right axis, X forward, and Z vertical, Cartesian positions transform as $[x, y, z] \to [x, -y, z]$ and Euler orientations as $[\text{roll}, \text{pitch}, \text{yaw}] \to [-\text{roll}, \text{pitch}, -\text{yaw}]$. In joint space, left and right joint vectors are swapped ($\boldsymbol{q}_{\text{left}} \leftrightarrow \boldsymbol{q}_{\text{right}}$), and joints with rotation axes in the sagittal (X–Z) plane additionally require a sign flip ($q \to -q$), while joints rotating about the lateral (Y) axis

TABLE V: Fine-tuning regime by component.

Component	Status	Loss / gradient source
Vision encoder (ViT)	Frozen	None
LLM backbone	LoRA	Autoregressive cross-entropy (subtask text / discrete tokens)
Interface (VLM $\rightarrow$ AE)	stop-gradient	No gradients flow back into LLM
Action expert (AE)	Hot (fully unfrozen)	Continuous flow-matching loss (L_{flow})

keep their sign, per the platform's URDF joint origin convention.

3) *Action flips.* The same kinematic reflection rules from step 2 are applied to the continuous action trajectory $A_{1:H}$. If the action space is normalized before training, the geometric reflection is applied before per-dimension normalization (or the signs are adjusted directly in normalized space).
4) *Language and subtask mirroring.* Since $\pi_{0.5}$ consumes subtask text tokens, we apply string-replacement maps to all high-level instructions and subtask labels, swapping directional tokens ("left" $\leftrightarrow$ "right", "leftmost" $\leftrightarrow$ "rightmost") and rotational tokens ("clockwise" $\leftrightarrow$ "counterclockwise"), e.g. "use left arm to hold bin and right arm to pick up block" $\rightarrow$ "use right arm to hold bin and left arm to pick up block."

c) Action representation.: Actions are normalized by mean and standard deviation rather than quantized with the FAST tokenizer scheme used natively by $\pi_{0.5}$. **[GAP: align our action normalization/quantization pipeline with the FAST-style scheme $\pi_{0.5}$ expects, or confirm the mean/std normalization is intentionally retained and document why.]**

d) Fine-tuning regime.: Following the Knowledge Insulation (KI) recipe, the vision encoder (ViT) is kept frozen; the LLM backbone is trained with LoRA on the autoregressive cross-entropy objective over subtask text and discrete (FAST) action tokens; a stop-gradient is applied at the VLM$\rightarrow$ action-expert interface so no gradients flow back into the LLM from the action expert; and the action expert itself is fully unfrozen and trained with the continuous flow-matching loss L_{flow} (Table V).

E. Policy Ablations

Each policy ablation is evaluated with 10 rollouts per task on the 12 in-domain tasks (120 rollouts per policy variant); results are reported in Section V-B. **[GAP: rollout accounting does not currently reconcile: the total is stated as 1,200 real-world rollouts, which is exactly 120×10, but the active policy variants below (A: 4, B: 2, C: 2, with D deprecated and excluded) sum to 8, not 10. Confirm whether Ablation A's four variants count toward this total or are excluded as a sanity check (see below), and recompute the total accordingly.]**

a) A — Baseline architectures.: π_0, π_0-FAST, $\pi_{0.5}$, and RICL. **[GAP: decide whether this comparison appears in the main paper or remains an internal sanity check, as originally noted in the planning document.]**

b) B — Effect of Advantage-Weighted Flow Matching (AWFM).: $\pi_{0.5}$ with and without AWFM. **[GAP: find and cite the appropriate AWFM reference.]**

TABLE VI: **VFE ablation results (placeholder).** Value error is mean absolute error between predicted and human-annotated (real-world) or ground-truth (LIBERO-100) progress value, lower is better.

Ablation	Condition	Real-world		LIBERO-100	
		In-domain MAE	OOD MAE	ID MAE	OOD MAE
A: Vision encoding	Native, stride 10	X.XX	X.XX	N/A	N/A
	Native, stride 20	X.XX	X.XX	N/A	N/A
	Patch-merger, stride 10	X.XX	X.XX	N/A	N/A
B: Annotation method	SARM annotation	X.XX	X.XX	N/A	N/A
	Sparse reward	X.XX	X.XX	N/A	N/A
	Manual + subtask	X.XX	X.XX	N/A	N/A
C: VFE comparison	Ours, no context (Section III-C)	X.XX	X.XX	X.XX	X.XX
	Ours (best A+B, IC-VFE)	**X.XX**	**X.XX**	**X.XX**	**X.XX**
	Robometer	X.XX	X.XX	X.XX	X.XX
	Robometer, fine-tuned	X.XX	X.XX	X.XX	X.XX
	TOPReward	X.XX	X.XX	X.XX	X.XX

TABLE VII: **Policy ablation results (placeholder).** Success rate (%) pooled over the 12 in-domain tasks, 10 rollouts per task.

Ablation	Condition	SR (%)
A: Baseline architecture	π_0	XX.X
	π_0-FAST	XX.X
	$\pi_{0.5}$	XX.X
	RICL	XX.X
B: AWFM	$\pi_{0.5}$ w/ AWFM	XX.X
	$\pi_{0.5}$ w/o AWFM	XX.X
C: MSA	$\pi_{0.5}$ w/ MSA	XX.X
	$\pi_{0.5}$ w/o MSA	XX.X

c) C — Effect of Morphological Symmetry Augmentation (MSA).: $\pi_{0.5}$ with and without MSA (bimanual left/right mirroring, following the augmentation recipe in Appendix D). GR-RL reports an 11.1% success-rate boost from an analogous mirroring augmentation [23].

Ablation D (effect of subtask annotations on $\pi_{0.5}$) has been deprecated and is excluded from the paper.

F. VFE and Policy Ablation Results

Table VI reports the IC-VFE ablation results from Section V-A, and Table VII reports the policy ablation results from Section V-B. The VFE table also reports the analogous LIBERO-100 VFE comparison (Section IV-A) alongside the real-world results, rather than as a separate table; LIBERO-100 was not used to evaluate the vision-encoding (A) or annotation-method (B) axes, so those rows are marked N/A in the LIBERO-100 columns.

G. Statistical Protocol

We do not run multiple seeds per configuration for the real-world evaluation; instead, background appearance is allowed to vary naturally between rollouts, and initial object placement is drawn from roughly the same distribution used during data collection, providing some environmental variation across repeated rollouts of the same task. (LIBERO-100 uses $k = 3$ seeded repetitions of the main comparisons; see Section IV-A.)

H. Evaluation Metrics, Robot Platform, and Analysis Plan

We report the following in Section V:

- **Overall success rate**, on all 24 tasks and split by in-domain vs. out-of-domain.

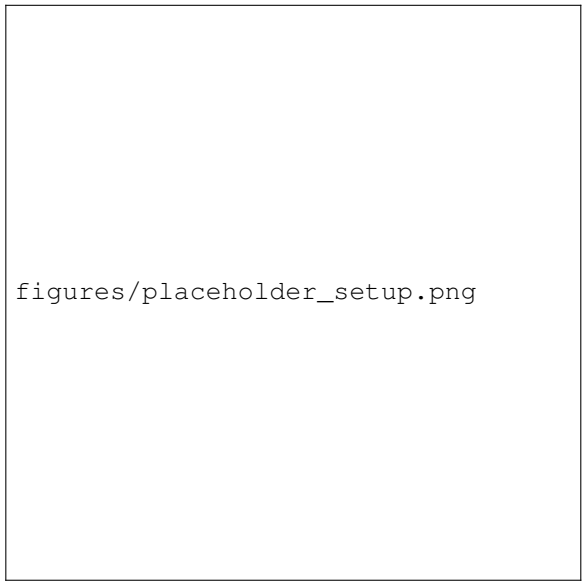


Fig. 5: **Experimental setup (placeholder).** Our YOR-based bimanual manipulator configuration, with fisheye wrist cameras and Agile-X NERO arms. **[GAP: placeholder figure – replace with a photo/diagram of the actual setup.]**

- **In-domain skill vs. out-of-domain object, same skill**: a breakdown restricted to task indices 1, 2, 4, 5, 8, 9, 10, 11, 12 (the same-skill/novel-object pairs, excluding the long-horizon composition pairs 3, 6, 7; see Section IV-B).
- **Success rate vs. number of context demonstrations**, from the ablation in Section IV-E.
- **Recovery success rate**: the fraction of rollouts in which the policy fails an intermediate subtask but recovers to complete that subtask, and separately, the fraction of those recoveries that go on to complete the full task. Subtask failure and recovery events are identified via manual annotation of rollout videos.

a) Success/failure judging protocol.: Each task has a fixed rubric against which raters judge rollout success; this rubric is applied consistently across all methods evaluated on that task.

b) Robot platform.: Evaluations are conducted on YOR [**?**], an open-source bimanual mobile manipulator. Our configuration uses fisheye wrist cameras on each arm (rather than YOR's stock iPhone-based gripper cameras) and Agile-X NERO arms, which have one additional degree of freedom relative to the Agile-X Piper arms used in the original YOR platform, plus a single head-mounted Zed camera (only one of its stereo channels is used). All camera images are resized to 640×480 at 30 fps. The action space is restricted to the two arms; YOR's omnidirectional base and vertical lift are not actuated during evaluation.

See Appendix G for the statistical (seeding) protocol.

c) Compute budget.: Each policy is trained on 4 H200 GPUs with a batch size of 128 for 100k training steps.

I. Qualitative Examples

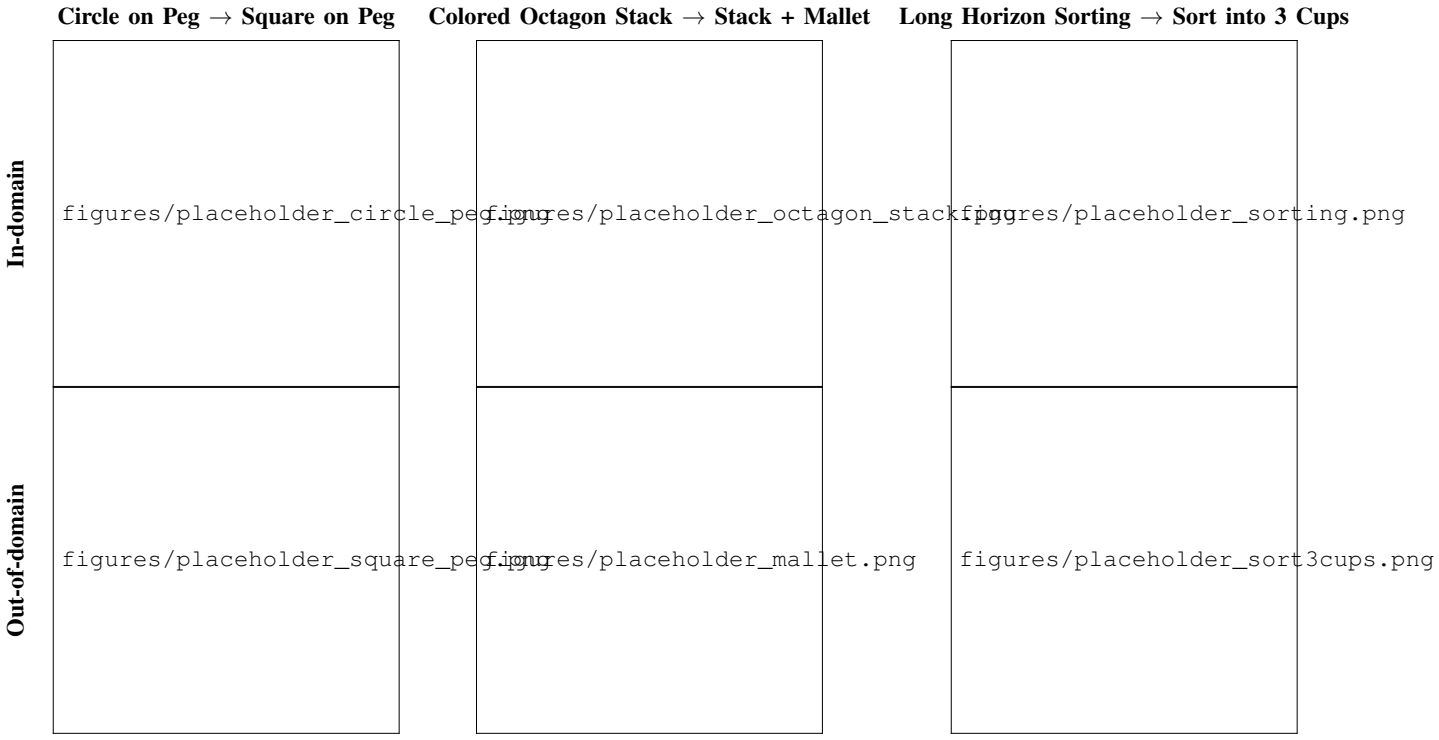


Fig. 6: **Qualitative in-domain / out-of-domain rollout comparison (placeholder).** Example rollout frames for task pairs 2, 3, and 6 from Table III, illustrating a task cousin (peg shape), a long-horizon composition (stack + mallet), and a long-horizon composition (sorting variant), respectively. **[GAP: placeholder figure – replace with real rollout frames once evaluation is complete; consider adding a failure-case column per Appendix E, Ablation A note on RICL failure modes.]**